

M53 — Lead student / group / list assignment from Multi-Patient Control

Phase: Phase 7 follow-on (post-M52, operator feedback) **Status:** **DONE** **Blocked by:** M5 (Multi-Patient Control), M22 (Per-Patient Console), M30 (existing lead_student_id) **Blocks:** none **Estimated effort:** 0.5 day

1. Purpose

"In the encounter Lead student needs to be able to add the name of the lead student, the name of a group, a list of students or the name of a single student to assign to a specific encounter or in several encounters. this may be more effective to locate in the Multi-patient control and then list the lead in the encounter or encounters as a reference for the instructor."

The existing M30 lead-student picker requires the lead to already be a registered student on the room's roster. That's fine when a single named student owns the bed, but the operator runs drills where:

- The lead hasn't joined yet (need to label before login).
- The lead is a **group** ("Team Alpha").
- Several students share a bed ("Alice, Bob, Charlie").

M53 adds a parallel free-text `lead_label` field that lives on each encounter, exposed via a new "■ Lead assignments" panel on the Multi-Patient Control dashboard. The panel supports per-encounter edits AND a bulk action that writes the same label to every checked bed in one click. The Per-Patient Console then shows the label read-only as a reference banner above the existing M30 picker.

2. What ships

2.1 Data model

`portal/control_session.py` — Encounter dataclass (= `ControlSession`) gains:

```
lead_label: str = ""
```

Independent of the M30 `lead_student_id: str | None`. Both can coexist. When the dashboard's `/api/room/state` is polled:

```
effective_lead_display = lead_label.strip() or lead_student_name or ""
```

So the free-text label wins for display, but falling back to the roster-picked name keeps M30 functional when no label is set.

2.2 API surface

Three new routes under existing `auth.require_*` guards:

Method	Route	Purpose
GET	<code>/api/room/lead_assignments</code>	Every encounter's current label + roster fallback in one call.
POST	<code>/api/encounter/{eid}/lead_label</code>	Set or clear ONE bed's label. Body <code>{"lead_label": "Team Alpha"}</code> or <code>""</code> to clear.

Method	Route	Purpose
POST	/api/room/lead_assignments	Bulk: {"assignments": [{"encounter_ids": [...], "lead_label": "..."}]}. Each assignment writes the same label to every listed bed. Unknown ids land in the response's unknown list.

Trims whitespace. No length cap, no validation — the operator's words are the source of truth.

2.3 Multi-Patient Control panel

New section in `portal/templates/control_room.html` (between Med carts and Nursing Station).

Pre-rendered server-side from the active room's encounters so it's visible on first paint:

```
■ ■ Lead assignments                                     ■
```

```
■ [ ] Bed 1      [Alice Pham          ] [Apply] [Clear]   saved ✓■
```

```
■ [ ] Bed 2      [Team Alpha           ] [Apply] [Clear]                   ■
```

```
■ [ ] Bed 3      [Alice, Bob, Carol    ] [Apply] [Clear]                   ■
```

```
■ Bulk apply:    [Team Alpha            ] [Apply to checked] [✓ All]■
```

Wire-up in portal/static/control_room.js's new wireLeadAssignments():

- Per-row Apply button POSTs the single-bed route.
- Per-row Clear button blanks the input and clicks Apply.
- Bulk Apply collects checked encounter ids, POSTs the bulk route.
- "Check / uncheck all" toggles every row checkbox in one click.
- No live re-render — operator-typed text shouldn't be overwritten

by a poll.

CSS in portal/static/control_room.css — indigo left-border accent to distinguish from green med-cart and green nursing-station panels.

2.4 Encounter-console reference banner

portal/templates/encounter_console.html — the "■ Lead student" card gets a hidden `.lead-label-ref` div above the M30 picker. When the encounter's `lead_label` is non-empty, the banner shows it with a "set from Multi-Patient Control" footer hint.

portal/static/encounter_console.js's pollState() now reads enc.lead_label from the state-poll response and calls _updateLeadLabelRef(label) to show/hide.

CSS in `portal/static/encounter_console.css` — same indigo accent as the dashboard panel for visual coherence.

3. Files touched

New

- `tests/v7/test_lead_assignments.py` — 19 acceptance tests.

- docs/module_guides/M53_lead_assignments.{md,pdf} — this guide.

Modified

- portal/control_session.py — lead_label: str = "" on ControlSession.
- portal/server.py — _encounter_summary surfaces lead_label + effective_lead_display; portal_room_get context includes encounters so the template can pre-render rows; three new routes (GET /api/room/lead_assignments, POST /api/encounter/{eid}/lead_label, POST /api/room/lead_assignments).
- portal/templates/control_room.html — new "■ Lead assignments" panel.
- portal/templates/encounter_console.html — .lead-label-ref banner inside the lead-student card.
- portal/static/control_room.js — wireLeadAssignments() + DOMContentLoaded hook.
- portal/static/control_room.css — .lead-assign-* styles.
- portal/static/encounter_console.js — _updateLeadLabelRef helper called from pollState.
- portal/static/encounter_console.css — .lead-label-ref styling.

4. Acceptance

Source: tests/v7/test_lead_assignments.py (19 tests).

1. **Dataclass** — ControlSession.lead_label defaults to "". 2. **Single-encounter set/clear/trim**: - POST with a value persists it; the state poll surfaces it. - Whitespace is trimmed. - Empty string clears. - Unknown encounter id → 404. 3. **Bulk multi-encounter**: - One assignment to N beds writes the same label to all of them. - Multiple assignments in one POST handle different labels per group. - Free-text accepts "Alice, Bob, Charlie" verbatim — no parsing. - Unknown ids land in unknown[], no 500. - Empty label clears every listed bed. 4. **GET endpoint** — every encounter listed; rows carry lead_label, effective_lead_display, encounter_label, lead_student_id. 5. **M30 coexistence** — when both lead_student_id (roster) AND lead_label (free-text) are set, the state poll's effective_lead_display shows the M53 label; with no label, it falls back to the roster pick's display name. 6. **Multi-Patient Control template** — panel renders one .lead-assign-row per encounter when a room is active; hidden when no room; pre-fills existing labels via value="..." on the input. 7. **Encounter console** — .lead-label-ref markup + footer hint present; JS state-poll reads lead_label + calls _updateLeadLabelRef; helper hides the banner when the label is empty. 8. **Dashboard JS** — wireLeadAssignments calls the single-bed route, the bulk route, and the check-all toggle.

All 19 pass. Full v7 suite **424 passed, 1 skipped, 0 regressions** (was 405 pre-M53).

5. Operator demo

1. Start a 3-bed room from the wizard. 2. Open /portal/room — the new "■ Lead assignments" panel appears between Med carts and Nursing Station. 3. Type "Team Alpha" in bed 1's input → Apply. Watch "saved ✓" appear next to the row. 4. Check beds 2 and 3, type "Team Bravo" in the Bulk apply input, click "Apply to checked encounters". Both rows' inputs mirror the label. 5. Drill into each encounter (/portal/room/encounter/{id}) — the ■ **Lead assignment** banner shows the right label above the existing M30 picker. 6. Clear bed 1's label → its banner disappears on the next poll.

6. Notes / non-goals

- **No notification cascade.** The Nursing Station and other student-facing surfaces don't yet show the lead label. Adding them is a future ticket if the operator wants the bedside students to see who's leading.
- **No schema migration.** `lead_label` is in-memory only — survives across pauses but dies with the server. The legacy `lead_student_id` already had no DB column so this is consistent. If the operator wants leads to persist across server restarts, a schema-v6 migration would add `lead_label TEXT` to `control_session` (out of scope here).
- **No history/audit log.** Each Apply overwrites the previous label. The chart-event log doesn't capture lead changes. If the operator wants an audit trail (who set what when), wrap `lead_label` writes in an `ehr_db.append_event(type="lead.assigned")` and add a debrief facet.
- **No length limit or HTML-injection escape on the server.** The browser-side `value="{{ enc.lead_label or '' }}"` is Jinja-autoescaped, but server-side echoes through API responses are JSON-encoded so they can't break HTML. If the operator wants a cap (e.g. 200 chars), add `lead_label[:200]` in `_handle_lead_label_set`.

Closes: the operator's M53 ask in full. Bulk assignment + per-encounter reference both deliver.